iOS Terminal Management Guide

The **iOS Terminal Management Guide** provides instructions for managing Ingenico Moby5500 devices using the TerminalManager class in the iOS SDK. It focuses on three key capabilities:

- **Retrieve Terminal Information**: Access details such as the terminal's model, serial number, and firmware version.
- **Check for Firmware Updates**: Determine if newer firmware versions are available for the device.
- **Perform Firmware Upgrades**: Update the terminal's firmware to maintain security and ensure it stays current.

Prerequisites

All operations require an active terminal connection, and developers must configure Bluetooth permissions in the app's Info.plist to support background connectivity and firmware updates. Before using these features, merchants must complete onboarding, obtain client credentials and access tokens, and connect their terminals as outlined in the Terminal Connection Guide. Each function includes detailed steps and callback methods to handle success and failure, enabling robust integration and real-time feedback.

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - iOS Merchant Authorization & Accounts Guide
- **Merchant Terminal Connection Guide:** This step is required to connect the Ingenico Moby5500 terminal to your iOS app.
  - iOS Terminal Connection Guide - Ingenico Moby5500

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Mandatory iOS Application Permissions

To enable terminal connection and management, specific iOS application permissions are mandatory. Therefore, the Merchant ISV app must include the required permissions in the app's Info.plist file to ensure proper functionality.

ON THIS PAGE

Prerequisites

Mandatory iOS Application Permissions

Retrieving Terminal Information

Checking for Available Firmware Updates

Performing Firmware Updates

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>App needs to connect with the Pos terminal to perform payments.
</string>
<key>NSBluetoothPeripheralUsageDescription</key>
<string>App needs to connect with the Pos terminal to perform payments.
</string>
```

Note: *To support firmware upgrades while the app is running in the background, your iOS application must declare the appropriate background mode capability by including the* ***<key>UIBackgroundModes</key>*** *entry in its* ***Info.plist*** *file. Without this configuration, firmware updates may fail when the app transitions to the background.*

Retrieving Terminal Information

To retrieve terminal information, ensure the terminal is properly connected and ready for communication. Once the connection is established, you can use the same TerminalManager instance to perform additional terminal operations.

Step 1: Create a TerminalManager object

```
let terminalManager = TerminalManager(merchantInfo: merchantInfo,
                                      terminalInfo: terminalInfo,
                                      merchantPosAppDetail:
merchantPosAppDetail)
terminalManager?.initialize(delegate: self)
```

Note: *Before retrieving terminal information, ensure the terminal is connected and online. It must be ready to communicate with the system for the operation to succeed.*

Step 2: Call the getTerminalInformation() Method

Once the terminal is connected, use the existing **TerminalManager** object to call the following method:

```
terminalManager?.getTerminalInformation()
```

Step 3: Implement the Success Callback

If the method call getTerminalInformation is successful, the following callback is triggered:

```
func onTerminalInformationAvailable(terminalDetails:
TerminalInformation) { }
```

The triggered callback returns a **TerminalInformation** object that includes:

firmwareVersion: Represents the firmware version of the Ingenico device.

batteryPercentage: Represents current battery level of Ingenico device.

terminalName: Represents the bluetooth name of the terminal device.

deviceID: Represents a unique identifier assigned to the Ingenico terminal device.

```
func onTerminalInformationFailed(error: String) { }
```

error: The error parameter provides a message describing the reason for the failure.

Checking for Available Firmware Updates

This process verifies whether the connected terminal is running the latest firmware by comparing its current version with the one available on the server. It ensures the device stays up to date with the latest features, enhancements, and security patches. If a newer version is detected, the system flags the need for an upgrade. The isUpgradeNeeded flag indicates whether an update is required, while availableVersion specifies the latest firmware version available for the terminal.

Step 1: Create and initialize the TerminalManager object

```
let terminalManager = TerminalManager(merchantInfo: merchantInfo,
                                      terminalInfo: terminalInfo,
                                      merchantPosAppDetail:
merchantPosAppDetail)
terminalManager?.initialize(delegate: self)
```

Note: *To verify the firmware version of the connected terminal, ensure it is properly connected and online. The check will only work if the terminal is ready for communication.*

Step 2: Initiate Firmware Version Check

Once the terminal is connected, use that TerminalManager object to initiate the firmware version check by calling the getAvailableFirmwareForUpgrade method.

```
terminalManager?.getAvailableFirmwareForUpgrade(terminalName,
                        currentFirmwareVersion,
                        bearerToken)
```

terminalName: Contains the terminal's device name

bearerToken: should be bearer token received from the MerchanAuthorizationService.

currentFirmwareVersion: Current firmware version of the connected terminal.

Step 3: Handling Successful Firmware Checks

If the check is successful, the **onGetAvailableFirmwareForUpgradeSuccess** callback is triggered. This callback provides information on whether an upgrade is needed, as well as the latest firmware version available for the device.

The callback parameters include:

availableVersion: The latest firmware version available on the server.

isUpgradeNeeded: A boolean flag indicating whether an upgrade is required.

Step 4: Handle Firmware Check Failure

If the firmware check fails, the onGetAvailableFirmwareForUpgradeFailure callback is invoked. This callback returns an error message detailing the reason for the failure.

```
func
onGetAvailableFormareForUpgradeFailure(status:TerminalConnectionStatus,
error: String) { }
```

status: TerminalConnectionStatus will be returned during failure.

- **TerminalConnectionStatus.getAvilableFirmwareForUpgradeFailed**: This is due to unexpected error.
- **TerminalConnectionStatus.getAvilableFirmwareForUpgradeFailedDueToInternet**: This is due to network issue.

error: The error parameter provides a message describing the reason for the failure.

Performing Firmware Updates

To keep the terminal updated with the latest firmware, initiate the firmware update process. This ensures the connected terminal downloads and installs the most recent firmware version available from the server. Before starting the update, make sure the terminal is properly connected and that valid firmware version details are provided—these are required for the process to begin successfully.

Step 1: Call the Firmware Update Method

```
terminalManager?.performFirmwareUpdate(terminalName,
                                       bearerToken,
                                       currentVersion)
```

terminalName: Contains the terminal's device name

bearerToken: should be bearer token received from the MerchanAuthorizationService.

currentVersion: Current firmware version installed on the device (e.g., "14.10")

Step 2: Handle Firmware Upgrade Progress

This callback is invoked during the update process and delivers real-time progress updates, including the terminal’s connection status and completion percentage.

```
func onFirmwareUpdateProgress(status, progress) { }
```

status: Represents the current TerminalConnectionStatus during the firmware upgrade process.

progress: Represents the percentage of upgrade completed.

Step 3: Handle Firmware Update Success

```
func onFirmwareUpdateSuccess() { }
```

Step 4: Handle Firmware Update Failure

This callback is triggered if the firmware update process fails. It provides the terminal status at the time of failure and a description of the error to assist with troubleshooting.

```
func onFirmwareUpdateFailed(status, error) { }
```

status: Terminal status will be returned during failure. Some examples are:

- **TerminalConnectionStatus.firmwareDownloadFailed**: This is due to server or network issue.
- **TerminalConnectionStatus.firmwareUpdateFailed**: This is due to terminal hardware or unexpected error.

error: The error parameter provides a message describing the reason for the failure.

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation